

Verteilte Systeme, Containerisierung

Vorwort:

Grundlage für diese Aufgabe ist die Präsentation «04_Ingress_Externe_Zugriffe»

Dauer: 90 Minuten

Besprechung: 30 Minuten

Total: 120 Minuten

Ziel: Ein gemeinsamer HTTP-Einstiegspunkt routet Dashboard, API und Health-Endpunkt. Zwei Dashboard-Pods machen die Verteilung sichtbar. Beim Löschen eines Pods bleibt der Dienst erreichbar.

SETUP

Startpunkt ist Ihr Projekt aus Block 3. Falls dieser Stand fehlt, verwenden Sie das öffentliche Foundation-Release v1.0.0. Für Block 4 benötigen Sie Zugriff auf das private Repository «vsc-dispatch-city-04-ingress», Release v1.1.1. Docker Desktop muss laufen; zusätzliche Werkzeuge wie make oder jq sind nicht nötig. Die Befehle sind für PowerShell sowie Bash/WSL/Git Bash/macOS angegeben.

Material:

- Foundation: <https://github.com/SwitzerChees/vsc-dispatch-city-03-foundation/releases/tag/v1.0.0>
- Block 4: <https://github.com/SwitzerChees/vsc-dispatch-city-04-ingress/releases/tag/v1.1.1>
- Dokumentation: <https://kubernetes.io/docs/concepts/services-networking/ingress/>

Aufgabe 1)

Baustein übernehmen und geplanten Stand sehen

Installieren Sie die kleine Block-4-Erweiterung in Ihrem eigenen Projekt und rendern Sie die Manifeste, bevor etwas im Cluster verändert wird.

- Klonen Sie das private Release einmalig neben Ihr Projekt-Repository.
- Führen Sie das passende Installationsskript im Wurzelverzeichnis Ihres Projekts aus.
- Rendern Sie das Overlay und suchen Sie Ingress-Klasse sowie Replikazahl.

PowerShell

```
git clone --branch v1.1.1 --depth 1 https://github.com/SwitzerChees/vsc-dispatch-city-04-ingress.git ..\vsc-dispatch-city-04-ingress
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
& "..\vsc-dispatch-city-04-ingress\install.ps1" -Target "."
kubectl kustomize ./deploy/overlays/block-04-ingress
```

Bash / WSL / Git Bash / macOS

```
git clone --branch v1.1.1 --depth 1 https://github.com/SwitzerChees/vsc-dispatch-city-04-ingress.git ../vsc-dispatch-city-04-ingress
../vsc-dispatch-city-04-ingress/install.sh .
kubectl kustomize ./deploy/overlays/block-04-ingress
```

ERWARTETES RESULTAT

Die Ausgabe enthält den Ingress «food-delivery», ingressClassName «traefik» und beim Dashboard replicas: 2.

Nachweis: Notieren Sie Ingress-Klasse und Dashboard-Replikazahl.

ingressClassName: traefik, dashboard replicas 2

Aufgabe 2)

Images bauen, importieren und deployen

Bauen Sie beide Images aus Ihrem Projekt. Dieser Wiederanlauf funktioniert auch dann, wenn der Namespace aus Block 3 nicht mehr existiert.

- Bauen Sie Control API und das aktualisierte Dashboard.
- Importieren Sie beide lokalen Images in den Multi-Node-Cluster «teko-k8s».
- Wenden Sie das Overlay an und warten Sie auf beide Rollouts.

PowerShell und Bash

```
docker build -t food-delivery-control-api:local --build-arg SERVICE=control-api -f build/go-service.Dockerfile .
docker build -t food-delivery-dashboard:local ./apps/dashboard
kubectl config use-context k3d-teko-k8s
k3d image import -c teko-k8s food-delivery-control-api:local food-delivery-dashboard:local
kubectl apply -k ./deploy/overlays/block-04-ingress
kubectl -n food-delivery rollout status deployment/control-api --timeout=180s
kubectl -n food-delivery rollout status deployment/dashboard --timeout=180s
kubectl -n food-delivery get deployment,ingress
```

ERWARTETES RESULTAT

control-api steht auf 1/1, dashboard auf 2/2. Der Ingress «food-delivery» verwendet die Klasse «traefik».

Nachweis: Halten Sie die drei READY-/CLASS-Werte fest.

control-api 1/1, dashboard 2/2, Ingress CLASS traefik

Aufgabe 3)

Ein Einstiegspunkt, drei Routen

Öffnen Sie den bereits installierten Traefik-Controller lokal und prüfen Sie Dashboard, API und Readiness über dieselbe Basisadresse.

- Warten Sie auf Traefik und starten Sie den Port-Forward in Terminal 1.
- Prüfen Sie die drei URLs in Terminal 2 und öffnen Sie danach das Dashboard im Browser.
- Falls Port 8080 belegt ist, verwenden Sie 8082:80 und ersetzen Sie den Port in allen URLs.

Terminal 1 - alle Systeme

```
kubectl -n kube-system rollout status deployment/traefik --timeout=180s
kubectl -n kube-system port-forward service/traefik 8080:80
```

Terminal 2 - PowerShell

```
(Invoke-WebRequest -UseBasicParsing http://localhost:8080/).StatusCode
(Invoke-WebRequest -UseBasicParsing http://localhost:8080/api/v1/snapshot).StatusCode
(Invoke-WebRequest -UseBasicParsing http://localhost:8080/health/ready).StatusCode
```

Terminal 2 - Bash / WSL / Git Bash / macOS

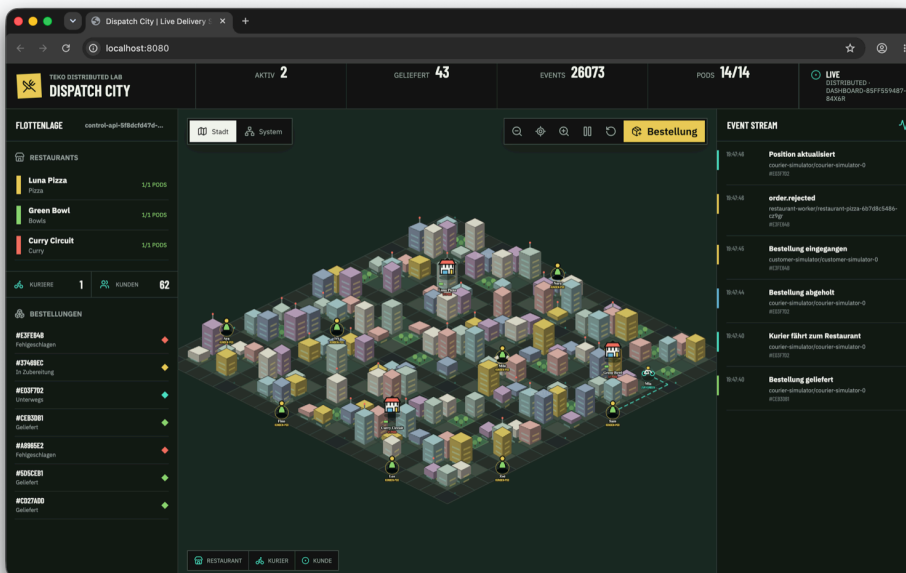
```
for url in /api/v1/snapshot /health/ready; do
  curl -s -o /dev/null -w "%{http_code} $url\n" "http://localhost:8080$url"
done
```

ERWARTETES RESULTAT

Alle drei Aufrufe liefern HTTP 200. Unter <http://localhost:8080/> ist die animierte Dispatch City sichtbar.

Nachweis: Notieren Sie die drei Statuscodes oder fügen Sie einen Screenshot ein.

200 /
200 /api/v1/snapshot
200 /health/ready



Aufgabe 4)

Load Balancing sichtbar machen

Der Service besitzt zwei bereite Dashboard-Endpunkte. Erzwingen Sie neue HTTP-Verbindungen und beobachten Sie, welcher Pod antwortet.

- Lassen Sie den Port-Forward aus Aufgabe 3 weiterlaufen.
- Senden Sie 20 kurze Anfragen mit Connection: close.
- Vergleichen Sie die Ausgabe mit der EndpointSlice des Dashboard-Services.

PowerShell

```
1..20 | ForEach-Object { (Invoke-RestMethod -Headers @{ Connection = "close" } http://localhost:8080/ui-instance).instance } | Sort-Object -Unique  
kubectl -n food-delivery get endpointslice -l kubernetes.io/service-name=dashboard -o wide
```

Bash / WSL / Git Bash / macOS

```
for i in {1..20}; do curl -s -H 'Connection: close' http://localhost:8080/ui-instance; echo; done | sort -u  
kubectl -n food-delivery get endpointslice -l kubernetes.io/service-name=dashboard -o wide
```

ERWARTETES RESULTAT

Es erscheinen zwei unterschiedliche Dashboard-Pod-Namen; die EndpointSlice enthält zwei bereite IP-Adressen.

Nachweis: Tragen Sie beide Pod-Namen ein und erklären Sie in einem Satz die stabile Rolle des Services.

dashboard-75dbdc8bb6-gtqv7

dashboard-75dbdc8bb6-ks6zt

EndpointSlice hat auch 2 IPs, 10.42.3.22 und 10.42.0.10

Service bleibt die feste Adresse und verteilt auf die Pods, die ready sind

Aufgabe 5)

Einen Pod löschen, ohne den Dienst zu verlieren

Löschen Sie eine Dashboard-Replik. Eine zweite Replik bedient weiter Anfragen, während das Deployment Ersatz erstellt.

- Beobachten Sie die Dashboard-Pods in Terminal 2 mit -w.
- Löschen Sie in Terminal 3 genau einen Pod und senden Sie danach 20 Anfragen.
- Warten Sie, bis das Deployment wieder 2/2 Replikas meldet.

Terminal 2 - alle Systeme

```
kubectl -n food-delivery get pods -l app.kubernetes.io/name=dashboard -w
```

Terminal 3 - PowerShell

```
$pod = kubectl -n food-delivery get pod -l app.kubernetes.io/name=dashboard -o  
jsonpath='{.items[0].metadata.name}'  
kubectl -n food-delivery delete pod $pod  
1..20 | ForEach-Object { (Invoke-WebRequest -UseBasicParsing http://localhost:8080/ui-instance).StatusCode }
```

Terminal 3 - Bash / WSL / Git Bash / macOS

```
pod=$(kubectl -n food-delivery get pod -l app.kubernetes.io/name=dashboard -o  
jsonpath='{.items[0].metadata.name}')
```

```
kubectl -n food-delivery delete pod "$pod"  
for i in {1..20}; do curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8080/ui-instance; done
```

Abschluss

```
kubectl -n food-delivery rollout status deployment/dashboard --timeout=180s  
kubectl -n food-delivery get pods -l app.kubernetes.io/name=dashboard
```

ERWARTETES RESULTAT

Die Anfragen bleiben erreichbar und liefern HTTP 200. Ein neuer Pod erscheint und das Deployment erreicht wieder 2/2.

Nachweis: Notieren Sie alten und neuen Pod-Namen sowie Ihre Beobachtung während des Ersatzes.

alt dashboard-75dbdc8bb6-gtqv7, neu dashboard-75dbdc8bb6-b7f84

ks6zt lief weiter, neuer Pod nach ca. 8s Running